REAL-TIME SCM & LOGISTICS 4K COMMAND CENTER

Complete System Architecture, Mathematical Foundations & Viva Voce Guide

Production Cloud Deployments: Vercel CDN Edge + Render Socket.io Gateway

1. The Core Problem Statement

In modern supply chain management and enterprise logistics, companies lose millions of dollars due to three systemic failures:

1. **The "Black Box" In-Transit Problem (Blind Logistics):** Once cargo leaves a central warehouse, dispatchers often rely on periodic driver phone calls or delayed EDI check-ins. If a truck faces highway gridlock or mechanical breakdown, dispatch centers learn about it hours later, missing critical delivery windows.
2. **The "Stockout vs. Overstocking" Dilemma (Suboptimal Inventory):** If a warehouse orders too little, stock runs out, assembly lines halt, and customer orders fail (costly stockouts). If a warehouse orders too much, huge amounts of working capital are frozen, and storage/holding costs skyrocket. Most legacy systems rely on arbitrary human guesses rather than mathematically verified demand forecasting.
3. **Fragmented, Laggy Data Silos:** Warehouse inventory managers, procurement teams, and transport dispatchers work on separate software systems. There is no single, unified, real-time command center where an inventory depletion immediately triggers a purchase order, updates a supplier, and tracks incoming transit on a single screen.

Our Solution

We engineered the Nexus SCM Real-Time 4K Command Center—a cloud-native, event-driven platform that integrates:

- Sub-second GIS Fleet GPS Telemetry over WebSockets.
- Automated Mathematical Replenishment using Reorder Point (ROP) & Economic Order Quantity (EOQ).
- An Interactive Kanban Procurement Pipeline bridging stock depletions to automated Purchase Orders.
- A High-Tech 4K Glassmorphism HUD providing dispatchers with instantaneous situational awareness.

2. High-Level Architecture Flow

```
+-------------------------------------------------------------------------------+
|                          FRONTEND SPA (Vercel CDN Edge)                       |
|  [Matrix Terminal Landing] -> [Authorization Gateway] -> [4K Command Center HUD]  |
|      - Leaflet GIS Dark Canvas Map (Truck Directional SVGs, Freight Polylines)    |
|      - Real-Time Chart.js Visual Stock vs ROP Curves & Donut Health Breakdown     |
|      - Interactive 4-Column Procurement Kanban Board                          |
+-------------------------------------------------------------------------------+
                                     | (Two-way WebSocket wss://)
                                     v
+-------------------------------------------------------------------------------+
|              REAL-TIME TELEMETRY & EVENT GATEWAY (Render Container)           |
|  Node.js / Express Server  |  Socket.io WebSocket Broker  | In-Memory Cache       |
|  - Telemetry Ingest: POST /api/telemetry/ingest                               |
|  - Event Ingest: POST /api/events/publish (inventory:alert, po:status_change)      |
|  - Round-Trip Ping/Pong Heartbeat Latency Monitor (4s ticker)                 |
+-------------------------------------------------------------------------------+
```

```
           ^                                          ^
           | (POST Ingest)                            | (Internal Webhook)
 +-----------------------------+            +-----------------------------------+
 |  IoT GPS Telemetry Engine   |            |  SERVERLESS API LAYER (Vercel)    |
 |  simulate_telemetry.js      |            |  - /api/inventory (ROP/EOQ Math)  |
 |  Highway coordinate waypoints |          |  - /api/purchase_orders (Kanban)  |
 |  Chicago -> Dallas -> LA    |            |  - /api/webhooks/notify_node.js   |
 +-----------------------------+            +-----------------------------------+
```

3. Deep-Dive: From Splash Screen to Command Center

View 1: Dynamic "Initiate System" Landing Page

- **The Visual Experience:** When you open the application, you are greeted by an animated cyan-green Matrix rain stream over a deep obsidian backdrop (#0b0f19) with a futuristic glitch title: NEXUS COMMAND CENTER.
- **Under the Hood:** It uses the native HTML5 Canvas API (<canvas id="matrix-bg"></canvas>). The canvas measures window.innerWidth and window.innerHeight. A string containing alphanumeric characters, mathematical operators, and symbols is split into vertical column drops. Every 33ms, the render loop paints a translucent rectangle (rgba(11, 15, 25, 0.05)) over the previous frame, creating an optical phosphor decay (fading trail effect), before stamping the next glyph in neon cyan (#00f3ff). Clicking "INITIALIZE SYSTEM" calls switchView("view-login") with zero browser reload.

View 2: Authorized Access Control Screen

- **The Visual Experience:** A glowing glassmorphism login panel with frosted glass blur (backdrop-filter: blur(16px)), neon borders, and authorized personnel verification.
- **Under the Hood:** Submits an asynchronous POST request to /api/auth. Verifies credentials (admin / password). Critical Performance Optimization: Once authentication succeeds, clearInterval(matrixInterval) is called to terminate the matrix canvas loop. This frees up 100% of GPU/CPU resources for rendering the 4K telemetry charts and the interactive GIS map.

4. Deep-Dive: The Live GIS Telemetry Map

The map is the operational heart of the Command Center, implemented in public/js/tracking.js using Leaflet.js.

1. The High-Tech "Dark-Matter" Styling

Standard OpenStreetMap tiles are brightly colored, which clashes with a 4K dark command room. Instead of paying for costly proprietary Mapbox keys, we used standard OpenStreetMap tiles and transformed them into a dark canvas using pure CSS hardware-accelerated filters:

```
#gis-map .leaflet-tile-pane {
  filter: invert(100%) hue-rotate(180deg) brightness(85%) contrast(120%);
}
```

This converts the map into a deep navy-black canvas where neon routes and markers pop out with crisp clarity.

2. Strategic Logistics Hubs

Warehouses are plotted at exact geographic coordinates:

- Chicago Apex Hub (ORD-01): [41.9742, -87.9073]
- Dallas SuperCenter (DFW-02): [32.8998, -97.0403]

- Ontario Depot (LAX-03): [34.0537, -117.5982]
- Newark Port Facility (EWR-04): [40.6895, -74.1745]

Each warehouse uses a custom L.divIcon displaying a pulsing emerald beacon (.custom-hub-marker). Clicking a hub opens a custom dark HUD popup showing warehouse code, operational name, and location.

3. Animated Vehicle Markers & Directional Heading

- Each vehicle is rendered using an SVG chevron arrow inside a vehicle-div-wrapper.
- Dynamic Orientation: When the truck turns, the SVG rotates dynamically to match the truck's actual bearing using inline CSS transforms: style="transform: rotate(${heading}deg);"
- Color-Coded Status: Neon Cyan (#00f3ff) for on-schedule vehicles ("In Transit"), and Crimson Red (#ef4444) for transport hazards or delays ("Delayed").

4. Polyline Highway Corridors

Active routes are traced across major US interstate corridors (e.g., I-80, I-35, I-10) using L.polyline. Routes are styled with semi-transparent cyan dashed strokes (dashArray: "6, 8"), displaying the planned trajectory between source and destination hubs.

5. How Live GPS Data Reaches the Map (The Streaming Pipeline)

1. **CLI Simulator:** simulate_telemetry.js reads GPS waypoints along freight routes and POSTs coordinates every 1.2s to Render.
2. **Render Gateway:** Caches vehicle coordinates in activeVehicleCache and fires io.emit("telemetry:update", payload).
3. **Browser HUD:** telemetryRadar catches the packet and invokes trackerMap.renderOrUpdateVehicle(data).
4. **Smooth Translation:** Existing marker position is updated with marker.setLatLng([lat, lng]) and rotated smoothly without redrawing the map or flickering.

5. Inventory Analysis & Mathematical Models

Rather than using arbitrary alerts, the system executes two classical industrial engineering formulas inside api/inventory.js:

1. Reorder Point (ROP) - "When to Order"

The Reorder Point represents the inventory level that triggers a replenishment order so that new stock arrives just as safety stock is reached:

```
ROP = (Daily Demand Rate * Supplier Lead Time) + Safety Stock

Safety Stock = Z * sqrt(Lead Time) * sigma_d
             = 1.65 * sqrt(Lead Time) * sigma_d  (for 95% Service Level)
```

- Daily Demand Rate (d): Units consumed per operational day.
- Lead Time (L): Number of days required for the supplier to manufacture and ship the items.
- Z-Score (Service Level): We chose Z = 1.65, which corresponds to a 95% service level, ensuring only a 5% statistical probability of stockout during supplier lead time.
- sigma_d: Standard deviation of daily demand.

2. Economic Order Quantity (EOQ) - "How Much to Order"

The EOQ (Wilson Formula) calculates the optimal order size that minimizes total inventory costs by finding the mathematical equilibrium between holding costs and ordering costs:

```
EOQ = sqrt( (2 * Annual Demand * Order Cost) / Holding Cost )
```

- D (Annual Demand): Total number of units required per year.
- S (Order Cost): Fixed administrative and shipping cost incurred every time an order is placed.
- H (Holding Cost): Annual cost of storing, insuring, and handling one unit in the warehouse.

3. Stock Health Classifications

- OPTIMAL (Emerald Badge): Current stock > ROP. Safe operational buffer.
- LOW_STOCK (Amber Badge): Current stock <= ROP. Replenishment needed.
- CRITICAL_REORDER (Pulsing Crimson Badge): Current stock < 70% of ROP. Urgent replenishment triggered.
- OUT_OF_STOCK (Solid Red Badge): Stock = 0. Critical disruption alert.

4. Chart.js Visual Telemetry

- Dual-Axis Bar & Line Chart: Displays on-hand inventory levels for every SKU (cyan vertical bars) contrasted directly against a horizontal red dashed threshold representing the Reorder Point. If any bar drops below the dashed line, it visually alerts the operator.
- Donut Chart: Renders the proportional health breakdown of the entire warehouse network (Optimal vs Low vs Critical).

6. The Automated Procurement Pipeline (Kanban Flow)

The automated reorder workflow links stock deductions directly to supplier purchase orders:

1. **Stock Deduction:** Operator or ERP deducts stock via POST /api/inventory?action=deduct.
2. **Automated Threshold Check:** System evaluates current_stock <= ROP. If true, status updates to CRITICAL_REORDER.
3. **Auto-Generation of PO:** System automatically creates a new Purchase Order with quantity = max(EOQ, 50).
4. **Internal Webhook:** /api/webhooks/notify_node.js sends events to Render (POST /api/events/publish).
5. **WebSocket Broadcast:** Render gateway emits inventory:alert and po:status_change to all connected HUD browsers.
6. **Reactive UI:** The Kanban board receives the new Draft card and the stock table highlights in red without page refresh.

The Kanban Stages

- Draft (Auto-Generated): Automatically spawned when ROP is breached with the exact EOQ batch quantity.
- Issued: Procurement officer verifies supplier pricing and issues the PO to the vendor.
- In Transit: The vendor dispatches freight; shipping coordinates bind to the GIS tracking map.
- Received: Shipments arrive at the warehouse loading dock, stock is replenished, and the item status resets to OPTIMAL.

7. How We Made the System Truly Dynamic

- Decoupled Event Gateway: Vercel handles edge assets, while Render maintains persistent bi-directional WebSockets (Socket.io). State changes broadcast instantly across the network.
- Reactive DOM Updates: When a WebSocket packet arrives (telemetry:update or inventory:alert), the browser dynamically updates only the affected DOM nodes or markers without re-rendering the entire page.
- Continuous Radar Ping-Pong: Every 4 seconds, the client sends a hud:ping timestamp to Render, which bounces back a hud:pong. The round-trip time updates the live HUD latency in milliseconds (e.g., 38ms).
- Simulation Engine: A background CLI runner (simulate_telemetry.js) feeds live GPS telemetry packets along real highway corridors into Render, causing vehicles to traverse the continental US in real time.
- Dynamic Service Binding: public/js/websocket.js supports dynamic connection strings via URL parameters (?node=...),

browser localStorage, or environment defaults, allowing the system to run in the cloud or on local networks.

8. Summary: How This Solves the Problem Statement

Problem in Traditional SCM	How Our Command Center Solves It	Technical Implementation
Blind In-Transit Freight	Sub-second GIS map updates showing exact vehicle position, speed, heading, and delays.	Leaflet.js + Socket.io GPS packet streaming over WebSockets.
Frequent Stockouts	Mathematical Reorder Point triggers replenishment before stock runs dry.	ROP = (d * L) + (1.65 * sqrt(L) * sigma_d) in /api/inventory.js.
Excess Holding Costs	Orders are batched at mathematically optimal cost curves.	Wilson EOQ Formula sqrt(2DS/H) calculates exact order size.
Delayed Procurement	Eliminates manual paperwork by auto-generating Draft POs the instant ROP is breached.	Automated event-driven PO creation linked to Kanban board.
Fragmented Systems	All data (fleet, stock, POs, latency) unifies on a single 4K command screen.	Glassmorphism SPA + Event Bus on Vercel & Render.

9. 2-Minute Viva Speech Script

"Respected Examiners, our project addresses the core challenge of modern supply chain management: lack of real-time visibility and delayed inventory replenishment.

We built the Nexus SCM 4K Command Center using a decoupled cloud architecture: our frontend and serverless APIs run on Vercel, while our real-time telemetry gateway runs on Render using Node.js and Socket.io.

When an operator logs into the Command Center, they see three synchronized layers:
1. First, our GIS Fleet Tracking Map built with Leaflet.js, which receives live GPS coordinates and directional heading over WebSockets from our simulation engine, rendering animated vehicle markers along transport corridors.
2. Second, our Inventory Intelligence Engine, which monitors stock levels against mathematically computed Reorder Points (ROP) using a 95% service level.
3. Third, our Automated Procurement Kanban Board. The moment an item's stock drops below its Reorder Point, the system automatically calculates the Economic Order Quantity (EOQ) to balance ordering and holding costs, generates a new Purchase Order, and broadcasts the alert over WebSockets to all connected command center screens without requiring a page refresh.

By combining WebSockets with classical operational management formulas, we eliminate blind transit spots and prevent both stockouts and overstocking."